Implementation Plan: Extended Cross-Format DNA Support

Overview

The **Text DNA** (normalized text hash) and **Visual DNA** (perceptual hash) pipelines cover common certify/verify paths (`pdf`, `docx`, Office, iWork via conversion, `txt`, `md`, images, etc.). This document tracks **what is already shipped**, what remains **backlog**, and **future** ideas (media fingerprints, OCR, dual-hash). It overlaps thematically with `post_mvp15_scale_survivability_and_batch.md` only where **Hamming / corpus** work touches perceptual DNA — that file is mostly scale/registry; **this** file is format + extraction + hashing.

Primary code today: `lib/text-dna.ts`, `lib/doc-convert.ts`, `lib/perceptual-dna.ts`, `app/actions/compound-dna.ts`, `app/actions/forensic.ts` (anchor path + `isDocument` guard).

Status (March 2026)

Area	State	Notes
Office / iWork → PDF (ConvertAPI, Netlify-safe)	**Shipped**	`lib/doc-convert.ts` — Word/Excel/PPT + iWork `pages` / `numbers` / `key`; tests in `lib/doc-convert.test.ts`.
Text DNA: PDF, DOCX, XLSX/XLS, PPTX native extractors	**Shipped**	`lib/text-dna.ts` (mammoth, exceljs, jszip/pptx XML); conversion-first for iWork; conversion fallback for many Office MIME types when needed.
Visual / perceptual path + page fingerprints	**Shipped**	`lib/perceptual-dna.ts`, `page_fingerprints`, parity tests — see North Star / verification tests.
Video keyframe fingerprint	**Not done**	Backlog (below).
Audio acoustic fingerprint	**Not done**	Backlog.

OCR (image → text DNA)	**Not done**	Phase 3 backlog.
Speech-to-text for audio/video	**Not done**	Phase 3 backlog.
Dual-hash (`strict_text_sha` / legal-grade punctuation)	**Not done**	No schema or `strict_text` in code yet; Phase 4 below.
RTF dedicated extraction	**Unclear / likely partial**	`text-dna` routes some `text/*` and `application/rtf` to plain text — **double-check** real RTF files.
HEIC	**Not done**	Env/deployment (`libheif` / WASM) still open.

What to double-check or discuss

1. **`application/msword`** → **`extractDocxText` (mammoth)** in `text-dna.ts` — legacy **binary** `.doc` may not extract reliably; ConvertAPI → PDF → pdf-parse is the robust path. Decide: document mammoth as “best effort only” or route `.doc` conversion-first like iWork.
2. **`isDocument` in `anchorToPolygon` / `forensic.ts`** — guard lists PDF/DOC/DOCX/XLSX; **pptx, iWork MIME types** may not be included in that boolean. **Verify** contentSHA requirements match every format you certify for production.
3. **Dual-hash promotion** — Adding `strict_text_sha` to Neon + anchor implies **contract/subgraph/migration** — treat as a **product + architecture** decision, not a silent ticket.
4. **Historical Wave / item references** (e.g. “item 22 Wave 3”) — superseded by current `doc-convert` + parity tests; use tests as source of truth.

The Backlog: Perceptual DNA for Media

To truly support Media (where a "Visual Match" matters even if the file size changes), we need to expand the DNA extractors.

Media Type	Current Status	What's Needed (Backlog)

Images	ACTIVE	Already uses Perceptual Hashing (Hamming distance) to allow minor edits/resizing.
Video	BACKLOG	Needs a "Keyframe Extractor." Take 5–10 screenshots of the video, hash them, and store a Video Fingerprint.
Audio	BACKLOG	Needs an "Acoustic Fingerprint" (like Shazam). Hash the frequencies so a match is found even if there's background noise.

The "Triple Lock" Path for Video/Audio

When we move Video and Audio out of the backlog, the Extractor Registry refactor (Task 3.1) makes it easy:

- **Binary Lock:** Still uses the `file_sha` (SHA-256).
- **Context Lock:** Still uses GPS/Timestamp/Scribe metadata.
- **Media Lock (The New Part):** Add `video-dna.ts` or `audio-dna.ts` to the `lib/` folder.
    - **For Video:** Extract a "Robust Fingerprint" using a library like ffmpeg.
    - **For Audio:** Generate a spectrogram hash.

The Sandwich Method: Where You Are

The Good: Verification fan-out already handles multiple match types (binary vs. perceptual). Images and PDFs use perceptual hashing where applicable.

The Reality: `forensic.ts` uses `isDocument` to **require** **`contentSHA`** (text DNA) for certain MIME types before anchoring — it does **not** block video/audio certification; those formats simply do not get the same “document must have text hash” guard. Video/audio still lack dedicated **content/media** DNA extractors (see backlog table above).

The Good: `lib/text-dna.ts` uses a **registry + conversion** pattern (`doc-convert`, conversion-first, fallback). When video/audio extractors exist, they can plug in similarly without rewriting the whole certify path.

Phase 1: Business Document Extensions (Low Effort, High Impact)

As of 2026 (shipped): `lib/text-dna.ts` implements native extraction for **PDF** (`pdf-parse`), **DOCX** (`mammoth`), **XLSX** (`exceljs`), **PPTX** (JSZip + slide XML), plus **plain text** and conversion-first / fallback via **`lib/doc-convert.ts`** (ConvertAPI) for many Office and iWork MIME types. Tests: `lib/text-dna.test.ts`, `lib/doc-convert.test.ts`.

The bullets below are the **original design notes**; treat the repo as source of truth.

1. Legacy Word Documents (`.doc`)

- **Challenge:** True **binary** `.doc` is not the same as OOXML; `mammoth` targets WordprocessingML (`.docx`).
- **As implemented:** `application/msword` is wired to **`extractDocxText`** (mammoth) in the extractor registry — **may fail** on legacy binary `.doc`. **Robust path:** ConvertAPI Word → PDF → `pdf-parse` (see `doc-convert` + `computeTextHashViaConvertedPdf`).
- **Optional enhancement:** Add **`word-extractor`** (or conversion-first for `.doc` only) if native binary `.doc` must be supported without conversion.

2. Spreadsheets (`.xlsx`, `.xls`, `.csv`)

- **Shipped (text DNA):** Tabular extraction via **`exceljs`** for modern Excel; conversion fallback where needed. Reading order and concatenation follow the implementation in `text-dna.ts`.
- **Remaining nuance:** Cross-format parity with a **CSV export** of the same sheet should stay covered by tests — add cases if gaps appear.

3. Presentations (`.pptx`, `.ppt`)

- **Shipped (text DNA):** PPTX XML slide extraction in `text-dna.ts`; visual path uses PDF conversion + perceptual pipeline where applicable.
- **Legacy `.ppt`:** Rely on conversion → PDF unless a native extractor is added.

Phase 2: Interoperability Ecosystems (Medium Effort)

These formats require specific handling due to proprietary structures or embedded formatting.

Netlify-constrained conversion endpoints (ConvertAPI)

Status: implemented — ConvertAPI is the external worker behind `lib/doc-convert.ts` (no LibreOffice on Netlify). The following endpoints match **current** code paths and parity tests; historical "item 22 / Wave 3" labels are **deprecated** — use tests + `doc-convert` as the contract.

Convert endpoint	Inputs	Role in current pipeline
Word → PDF	`.docx`, `.doc`	PDF buffer for text (`pdf-parse`) and visual (`renderPdfPageAndHash` / perceptual path).
Excel → PDF	`.xlsx`, `.xls`	Visual fingerprinting for spreadsheets when native render path needs a PDF.
PowerPoint → PDF	`.pptx`, `.ppt`	`robustFP` / visual path for decks.

iWork suite (implemented in `doc-convert`):

- Pages → PDF (`application/vnd.apple.pages`)
- Numbers → PDF (`application/vnd.apple.numbers`)
- Keynote → PDF (`application/vnd.apple.keynote`)

Forensic utilities (advanced verification / backfill support):

- PDF -> Image (`png`/`jpg`) for faster page-level fingerprint backfill workflows.
- Async/webhook conversion mode for long-running jobs (for example large PDFs) so Netlify functions do not wait on full processing and risk timeouts.

Guardrail: native parsing remains useful for text extraction, but Visual DNA for Office/iWork still requires PDF conversion in the render-first path.

1. Rich Text Format (`.rtf`)

- **Challenge:** RTF contains visible text mixed with hidden markup tags (e.g., `\rtf1\ansi`). Reading it as plain text ruins the hash.
- **Implementation:**
  - Install `rtf-parser` or `textract`.
  - Strip the RTF control words and extract only the visible Unicode text stream.
  - Route through the standard `canoicalizeText` pipeline.

2. Apple iWork Suite (`.pages`, `.key`, `.numbers`)

- **Challenge:** Proprietary packages; native Node parsing is brittle.

- **Shipped:** ConvertAPI pages / numbers / key → PDF in `lib/doc-convert.ts`, then standard PDF text + visual pipelines. Provider swap remains isolated in that module.
- **Fallback:** If quota or provider fails, user guidance to export to PDF/DOCX is still valid product copy.

3. Apple HEIC Images (`.heic`)

- **Challenge:** The `sharp` image library requires special system-level libraries (`libheif`) to decode `.heic` files, which often aren't present in standard Docker/Vercel environments.
- **Implementation:**
  - Configure the deployment environment (e.g., custom Dockerfile) to install `libheif` and a `heif`-enabled version of `sharp` or `heic2any`.
  - Or, use a lightweight WASM-based HEIC-to-JPEG converter before passing the buffer to the Visual DNA pipeline.

Phase 3: AI-Driven Extractions (High Effort, Massive Capability)

This phase introduces machine learning pipelines to extract Text DNA from formats that don't natively contain selectable text.

1. Optical Character Recognition for Images (`.png`, `.jpg`, Video Frames)

- **Challenge:** An image of a physical contract provides a Visual DNA, but cannot currently match the Text DNA of its original PDF.
- **Implementation:**
  - Integrate an OCR engine (like `Tesseract.js` for local execution, or an API like Google Cloud Vision for higher accuracy).
  - When an image is certified, run OCR to extract the text, run it through `stripFormatting` and `canonicalizeText`, and store a *Text DNA* alongside its Visual DNA.
  - *Impact:* Allows physical printouts (photos) to be verified directly against digital master PDFs.

2. Speech-to-Text for Audio/Video (`.mp3`, `.wav`, `.mp4`)

- **Challenge:** Audio files currently only get a binary File DNA.

- **Implementation:**
    - Integrate an audio transcription model (e.g., OpenAI's Whisper API).
    - Transcribe the audio buffer into text.
    - Pass the transcript into the standard Text DNA pipeline.
    - *Impact:* Allows audio recordings to be verified against written transcripts or meeting minutes.

Phase 4: The Dual-Hash Strategy (Precision Tampering vs Formatting Robustness)

Currently, Text DNA strips all punctuation, resolving only Unicode letters and numbers (`/[^\p{L}\p{N}]/gu`). This provides **high cross-format compatibility** (e.g., surviving a straight-quote `"` to curly-quote ″ conversion by a PDF engine), but creates a **blind spot for precision tampering** (e.g., tampering `$10.00` to `$1000` evaluates as the same `1000` sequence).

The Implementation: Dual Hashes

To serve legal/financial use-cases while maintaining cross-format resilience, the pipeline must compute *two* distinct Text DNA hashes simultaneously:

1. **Robust Hash (Current Logic):**
    - Strips whitespace, layout, and all punctuation.
    - Used to confirm the substantive words match, regardless of formatting or export-engine quirks.
2. **Strict Hash ("Legal-Grade"):**
    - Strips only whitespace/layout (`\s+`), but *preserves* punctuation and symbols (decimals, commas, currency signs).
    - Prevents number-shifting tampering (`1,000.50` vs `100,050`).

Verification Outcomes

When verifying, the UI will report the relationship between these two hashes:

- **Perfect Match:** Both hashes match (The document is mathematically identical in content and punctuation).
- **Substantive Match (Formatting Altered):** Only the Robust Hash matches (e.g., a Word-to-PDF export changed a hyphen type, but the words are identical. UI shows a warning: *"Content matches, but punctuation/formatting differs. Review manually."*)
- **No Match:** Neither hash matches (The document was tampered with, e.g., words changed or decimals shifted).

Implementation Plan (not started — backlog)

1. **Extend DNA Pipeline:** Modify `text-dna.ts` to export two hashing functions: `computeRobustTextHash` and `computeStrictTextHash`. *(Today: single robust/canonical path; no `strict_text` in TS.)*
2. **Schema Update:** Add `strict_text_sha` to the `forensic_records` database schema.
3. **Certification Flow Update:** Update `compound-dna.ts` and `forensic.ts` to compute both hashes and anchor/index both values on Polygon and Neon.
4. **Verification Logic Update:** Update `verification.ts` to query for both hashes.
5. **UI Update:** Update the `RealityCapture` and `RealityCheck` UI components to display the new "Substantive Match" status banner when only the robust hash matches.

Implementation Plan Step 2 (Database Schema) will require a Migration. Since you are adding strict_text_sha, your current records in Neon/The Graph won't have this field. You'll need to decide if you want to "Backfill" those records or just allow old records to report N/A for the Strict Hash (your 5.1 Legacy Support task).

Testing & Validation Requirements (Dual-Hash)

To ensure the Dual-Hash strategy works correctly, the following automated test cases must be added to `lib/text-dna.test.ts`:

1. **Precision Tampering Test:**
   - **Input A:** `$1,000.50`
   - **Input B:** `$100,050`
   - **Expected:** Robust Hash matches (FAIL condition missed), Strict Hash NO MATCH (TAMPER CAUGHT). Final Verdict: **No Match**.
2. **Benign Formatting Test (Cross-Format Export):**
   - **Input A:** `The company "Authen.ai"`
   - **Input B:** `The company “Authen.ai”` (curly quotes)
   - **Expected:** Robust Hash MATCHES, Strict Hash NO MATCH. Final Verdict: **Substantive Match (Review formatting)**.
3. **Layout Noise Test (PDF vs DOCX):**
   - **Input A:** `Line 1 \n\n Line 2`
   - **Input B:** `Line 1 \t Line 2`
   - **Expected:** Both Robust and Strict Hashes MATCH. Final Verdict: **Perfect Match**.

Canonical vs Supporting DNA Policy

Authen.ai anchors a compact canonical certification record on Polygon. Additional fingerprints and page-level retrieval keys are stored off-chain as discovery and matching aids. Verification follows a discover-off-chain, verify-on-chain model.

Core rule: Only anchor what must be independently provable. Index everything else.

What is anchored on Polygon (canonical)

Value	Role
`fileSHA`	File DNA — exact binary identity
`contentSHA`	Text DNA — current canonical text hash (robust)
`robustFP`	Visual DNA — primary perceptual hash
`metadata`	Compressed forensic envelope

What is off-chain only (supporting / retrieval)

Value	Role
All-page visual fingerprints (item 23)	Every page indexed in `page_fingerprints`; ranking/aggregation at retrieval
DOCX-rendered robustFp (item 22)	Fallback visual fingerprint
`strict_text_sha` (Phase 4)	Future canonical target for Text DNA; robust hash becomes supporting
Robust text hash (after strict ships)	Cross-format retrieval fallback
Video / audio fingerprints (Phase 3)	Media retrieval — anchor only if media certification becomes distinct product surface
Search indexes (`whisper_index`)	Operational discovery layer

Implementation guardrail

When adding new DNA values from Phase 1–4:

- New extractors feed into existing categories (File, Text, Visual) — do not multiply categories prematurely.
- New values default to **off-chain/index** unless explicitly promoted to canonical.
- Promotion to canonical requires: schema migration, contract update, subgraph mapping, and backfill decision.

General Testing Requirements

For each new file format added, a corresponding automated test must be written in `lib/text-dna.test.ts` to ensure:

1. **Intra-format stability:** Repeated extractions of the same file yield the same hash.
2. **Cross-format parity:** Extracting text from an `.xlsx` yields the *exact same Text DNA hash* as extracting from a `.csv` export of that same sheet.

Reality testing and calibration plan

Goal

Prove and bound what the product actually does so **NORTH_STAR.md**, **technical_assumptions_limitations_and_claims.md**, **docs/INVESTOR_BRIEF.md**, **arup_deepfake_case_study_future_use_case.md**, and **cross-format-dna-support.md** stay **aligned with measured reality**—especially Visual DNA (probabilistic), Text DNA (normalization blind spots), and registry-scoped negatives.

Treat **pre-pilot-validation-plan.md** as the **detailed benchmark spec** (V1–V8, corpus, transformation matrix, two-layer metrics). This plan is the **execution-oriented test plan** that maps claims → evidence → calibration actions.

```
flowchart LR
  subgraph claims [Claim sources]
    NS[North Star]
    TA[Technical assumptions]
    CF[Cross-format DNA]
    AR[Arup case study]
    IB[Investor brief]
  end
  subgraph tests [Test layers]
    REG[Regression and parity]
    CORP[Corpus plus transforms]
    MET[Metrics and thresholds]
    RED[Red team and stress]
  end
  subgraph out [Outputs]
    CAL[Threshold and policy]
    DOC[Doc and UI copy updates]
  end
  claims --> tests
  tests --> out
```

Ownership legend

Owner	Examples
You / team	Approve corpus sources; certify documents; run real WhatsApp/device captures; pick ε and policy from charts; staging secrets; pilot scope; final copy and legal/comms sign-off
Agent / eng	Phase 0 automation; traceability markdown; runner + logging code; plots/tables from manifests; doc PRs from agreed numbers; CI wiring and test fixes

---

Agent vs human checklist (chronological)

1. **You / team** — Set pilot scope, target environment (staging / testnet), and privacy rules for the corpus (no uncleared customer PII).
2. **Agent / eng** — Phase 0: run npm run test, test:actions, test:parity-slo, test:parity-acceptance (and stability script); fix regressions; keep smoke-verify path documented.
3. **You / team** — Provide or approve subgraph-backed certified fixture for smoke verify; confirm credentials/URLs for staging runs.
4. **Agent / eng** — Phase 1: create claim-to-test traceability sheet (markdown or linked spreadsheet structure) and link it from the pre-pilot plan or docs/.
5. **You / team** — Review traceability rows; confirm which claims are in scope for the first benchmark wave.
6. **You / team** — Approve corpus sources; certify originals (RealityCapture) or approve a synthetic corpus; stratify by document class.
7. **Agent / eng** — Phase 2: implement transformation runner, ground-truth manifest schema, and logging (e.g. match_type_top, Hamming); automate what can run headlessly (re-encode, resize, scripted crops).
8. **You / team** — Produce real-channel variants (WhatsApp forward, device screenshot, print-scan) per transformation matrix; register files in the manifest.
9. **Agent / eng** — Run benchmark batches, aggregate results, generate threshold curves and Layer 1 / Layer 2 FP–FN tables (Phase 3 inputs).
10. **You / team** — Choose default ε, optional per-class thresholds, and whether to change PDF_RENDER_SCALE based on charts; record decision (ADR or pre-pilot plan update).
11. **Agent / eng** — Phase 4: execute scripted near-duplicate, adversarial, and decision-policy checks; refresh metric tables.
12. **You / team** — Review stress/red-team outcomes; decide product stance on edge cases (templates, collisions, “no record” messaging).
13. **Agent / eng** — Phase 5: open doc PRs for NORTH_STAR, INVESTOR_BRIEF, cross-format DNA, technical assumptions, and Arup footnotes using **numbers and wording you approved**.
14. **You / team** — Final review of external-facing claims; optional counsel/comms pass.

Phase 0 — Regression baseline (always-on)

Purpose: Ensure shipped logic does not drift before/while you run expensive real-world runs.

Owner	Examples
You / team	Approve CI policy (what must stay green); provide env for manual smoke verify when automation cannot reach subgraph
Agent / eng	Run and maintain test/parity/smoke commands; add or fix unit tests when format work touches extractors

Activity	Where / how
Unit and action tests	npm run test, npm run test:actions — especially app/actions/verification.ts / app/actions/verification.test.ts (North Star: no match without chain-backed seals).
Parity / SLO gates	npm run test:parity-slo, npm run test:parity-acceptance (and stability script per package.json).
Smoke verify	scripts/smoke-verify-file.sh + scripts/fixtures/README.md — end-to-end API + subgraph path on a known certified fixture.
Conversion / text DNA	Extend or run lib/text-dna.test.ts, lib/doc-convert.test.ts when touching formats listed in cross-format backlog.

Claim alignment: “Chain-backed match,” conversion-backed paths, and format coverage statements in cross-format doc must **not contradict** green CI for the above.

Phase 1 — Claim-to-test matrix (traceability)

Use a simple **requirements-style table** (spreadsheet or markdown appendix) so every external claim has an ID and a test:

Claim theme	Source doc	What "reality" must show	Primary signals
Five-minute, independent verify	North Star / Investor	Time-to-result UX + automated timing on representative uploads	E2E or scripted verify latency
Screenshot/compressed/ forward **possible**, not guaranteed	North Star / technical assumptions	Hit rate vs transformation type; explicit "no match" rate	Visual + Text + File paths; match_type_top in logs
Visual = similarity, not crypto proof	Technical assumptions	Visual matches: Hamming distribution; adversarial/ near-duplicate behavior	[PERCEPTUAL_HAMMING_THRESHOLD](lib/perceptual-dna.ts), PDF_RENDER_SCALE
Text DNA cross-format	Cross-format / North Star	Same logical doc → same contentSHA where promised; document failures	Parity tests + real DOC/PDF/ Office/iWork samples
Registry-negative scoped	Arup / Investor / technical	No match ⇒ "not in **this** registry," not "never issued"	Copy checks + negative corpus (uncertified but plausible)
Arup: no video/deepfake	Arup case	Manual QA / doc audit only	Scope checklist

isDocument / contentSHA requirements	Cross-format “double-check”	Certify path rejects or warns per product policy for each MIME you market	Matrix test per MIME

Deliverable: **one traceability sheet** linked from [pre-pilot-validation-plan.md](.cursor/plans/pre-pilot-validation-plan.md) or docs/ (your choice when implementing).

Phase 2 — Corpus and transformation benchmarking (empirical core)

This is **V1–V3 + V4** from the pre-pilot plan, executed with discipline:

1. **Corpus (V1):** Certified originals by document class (press release, memo, disclosure, deck, spreadsheet, scan-like PDFs). Start smaller than 1,000 if needed, but **stratify** by class so metrics are not dominated by one type.
2. **Transformation runner (V2):** For each original, generate labeled variants: WhatsApp-style re-encode, screenshot from viewer, crop, blur, re-save, print-scan, annotation, round-trip PDF↔Office where relevant. **Ground truth:** variant → original_file_sha (or stable corpus ID).
3. **Instrumentation:** For each run, record: match / match_type_top (from [verification_log.metadata](app/actions/verification.ts)), Hamming distance when visual, and **candidate rank** if you surface multiple seals.

Code anchors: [verifyCompoundDNA](app/actions/verification.ts), [getHammingDistance](lib/perceptual-dna.ts), [PERCEPTUAL_HAMMING_THRESHOLD = 3](lib/perceptual-dna.ts), [PDF_RENDER_SCALE = 1.0](lib/perceptual-dna.ts).

Claim alignment: North Star / Investor “verifiable after screenshot, compressed, forwarded” becomes a **reported hit rate by transformation**, not a boolean slogan.

Phase 3 — Calibration (thresholds and policy)

V3 threshold curves + V4 FP/FN from pre-pilot plan:

- Sweep ε (e.g. 0–10) **per transformation family** and optionally **per document class**; plot ROC-style tradeoffs using corpus labels.
- Separate metrics: **Layer 1** (Visual retrieval: correct candidate in top-k?) vs **Layer 2** (end-to-end match=true with chain seal) — already defined in pre-pilot § "Two layers to measure."
- Decide: single global ε vs per-class ε (may require config + code change later; plan the **measurement first**).

Outputs:

- Recommended default ε and PDF_RENDER_SCALE (or confirm current baseline).
- Written **operating envelope**: "Under transformations T1..Tk and document classes C1..Cn, observed FP/FN are …"

Phase 4 — Stress, red team, and scope boundaries (V5–V8)

Milestone	Validates
V5 Near-duplicate / same-template	Investor / North Star not overstating "closed world" discrimination
V6 Adversarial visual tweaks	Technical assumptions (perceptual hash limits)
V7 C2PA / watermark comparison	Technical assumptions doc (complement, not replace) — optional if bandwidth
V8 Decision policy	When UI says File vs Text vs Visual vs no record — aligns with [getPublicVerificationBannerUi](lib/verification-record-display.ts)

Arup case study: Re-verify after Phase 2–4 that table rows still match **measured** false-match and false-non-match behavior for "forged PDF" scenarios; update wording if data shows edge cases (e.g. template collision).

Phase 5 — Documentation and product copy pass

After each calibration slice:

- Update **technical_assumptions_limitations_and_claims.md** with **measured** ranges or “pending” explicitly.
- Update **cross-format-dna-support.md** “Status” table with **evidence** (test file names, corpus notes) for .doc, RTF, HEIC, isDocument list, etc.
- Adjust **NORTH_STAR.md** / **INVESTOR_BRIEF.md** only where numbers or boundaries changed.
- Keep **arup_deepfake_case_study_future_use_case.md** strictly scoped; add a footnote “validated against corpus date …” when you have data.

Roles, environments, and artifacts

- **Environments:** Staging with real subgraph + Neon (or testnet parity); document which chain and indexer revision produced the report.
- **Artifacts:** Versioned **corpus manifest** (JSON/CSV), **run logs** (hash, transformation, outcome, Hamming), **summary notebook or markdown report** (threshold curves, tables by class).
- **Privacy:** No customer PII in shared corpus; use synthetic or cleared real docs.

Suggested sequencing (practical)

1. Phase 0 green + traceability sheet skeleton.
2. Small pilot corpus (e.g. 30–50 docs × 5 transforms) to validate **runner + metrics pipeline** before scaling to V1 scale.
3. Full V1–V4 → calibration decision.
4. V5–V8 + doc pass.

Out of scope for this test plan (explicit)

- New extractors (video/audio/OCR) — tracked in cross-format backlog; testing plan **documents gaps**, does not implement them.
- Legal sign-off — outputs support counsel review, not replace it.

Holistic QA test stream (certify, verify, cross-account)

Deliverable

After you approve this plan, add one markdown file (suggested path: docs/QA_E2E_TEST_STREAM.md) that testers can run top-to-bottom. It should reference **deployed testnet/staging** (or localhost) URLs, **Clerk orgs**, and **artifact slugs** as fill-in fields at the top.

Roles: what the agent does vs what you and your team do (chronological)

Use this as a handoff timeline so nobody expects the agent to click through Clerk or sign transactions on your behalf.

1. **Draft the QA playbook (post-approval)** — **Agent:** Create docs/QA_E2E_TEST_STREAM.md from this plan (streams, checklists, verdict semantics, optional mermaid). **Team:** Skim for accuracy; fill in real base URLs, org display names, and who plays Account A / B / C.
2. **Freeze the target environment** — **Team:** Confirm which deployment is under test (e.g. production testnet vs preview), that Polygon Amoy + Neon + subgraph are wired, and that recent deploys finished. **Agent:** Not involved unless you ask for a readonly config/code sanity check.
3. **Optional automated pre-flight** — **Team:** Run npm run smoke:check and, with dev server up, npm run smoke:verify per README.md / scripts/fixtures/README.md. **Agent:** Only if you paste errors — help trace failures against code paths (no live signing or RPC from the agent).
4. **Provision people and workspace shape** — **Team:** In Clerk, ensure orgs exist (authen-testnet + separate org for “random person X”), admins, and team/client **Membership Assignments** so Accounts A and B pass the workspace gate and Account C can be used for no-access. **Team:** Stage test files (original, derivative, unrelated). **Agent:** Does not manage Clerk or your wallet keys.
5. **Execute manual streams 0–8** — **Team:** Run every stream in order on real browsers (including incognito for public URLs); capture slugs, tx hashes, screenshots, and pass/fail. **Cross-org streams are human-only:** switching orgs and verifying **Official** vs **External** requires real sessions. **Agent:** Does not execute UI flows against your deployment.
6. **Triage failures** — **Team:** Reproduce, note steps, collect network/console errors. **Agent (on request):** Help interpret behavior against lib/verdict.ts, lib/workspace-gate.ts, middleware public routes, etc.; suggest additional cases to add to the doc.

7. **Sign-off and doc hygiene — Team:** Complete the sign-off table (mandatory row for cross-org); decide release readiness. **Agent (on request):** Update the QA doc with new edge cases or clearer steps discovered during the run.

Summary: The agent produces and refines the **written test stream** and can help **debug from code** when you bring evidence. Your team **owns environment, accounts, wallets, manual execution, and the release decision**.

Product surface to cover (grounded in repo)

Area	Route / entry	Notes
Landing	app/page.tsx redirects to /control-center	Anonymous users hit Clerk protection on dashboard
Certify (RealityCapture)	/certify + workspace ?teamId= / ?clientId=	Gated by lib/workspace-gate.ts: org members need at least one allowed **client** or they land on /workspace/no-access
Verify (RealityCheck)	/realitycheck	Same workspace gate as certify
Public certification	/certification/[slug] (alias /certify/[slug])	**Public** in middleware.ts
Public verification	/verify/[slug]	**Public**
Embed	/embed/[slug]	**Public**, compact iframe-oriented UI
Settings / org	/setup	**Protected**; issuer, gate targets, teams/clients, assignments
Scribe	/scribe	**Protected**; org logbook
Control Center	/control-center	**Protected**; defense score, gates, conversion usage, Hamming telemetry
APIs	POST /api/certify/dna, POST /api/verify/file	Certify path uses Clerk orgId for metering/scope (app/api/certify/dna/route.ts); verify persists attempts and derives badge using verifier org (app/api/verify/file/route.ts + lib/verdict.ts)

Verdict semantics for cross-org QA (critical): lib/verdict.ts — same certifier scribe_id as verifier's Clerk orgId → **Official**; different org (or missing verifier org) → **External**. Cross-org success means **match + correct External (or Official when same org)**, not "failed verify."

Preconditions (document header checklist)

- **Environment:** Polygon **Amoy** RPC, Neon DB, subgraph URLs as in .env.example / deployment secrets.
- **Accounts:** Minimum **three** Clerk identities:
  - **Account A** — authen-testnet (or primary certifier org): admin + workspace (team/client assignment).
  - **Account B** — separate org ("random person X"): at least one client assignment so /certify and /realitycheck are not blocked by workspace gate.
  - **Account C** — optional: member of A or B **without** team/client assignment (for no-access UX).
- **Test assets:** Original file for certify; optional derivative (screenshot/recompress) for visual/text DNA cases; unrelated file for expected no-match.
- **Automation hook (optional):** Document existing npm run smoke:check and npm run smoke:verify from README.md / scripts/fixtures/README.md as **pre-flight**, not a substitute for multi-account UI QA.

Test streams (run in order)

Stream 0 — Access and navigation smoke

1. **Anonymous:** Open site root → expect redirect to /control-center and Clerk sign-in / org selection flow.
2. **Signed in, org selected, no workspace:** Use Account C → navigate to /certify or /realitycheck → expect redirect to [/workspace/no-access](app/workspace/no-access/page.tsx) with coherent copy and links to Settings / return path.
3. **Signed in with workspace:** A and B → /certify and /realitycheck load without gate redirect.

Stream 1 — Certify (Org A)

1. As **Account A** on /certify, seal a **PDF** (and optionally **image** / **DOCX** if conversion path matters for you).
2. Confirm success UI: tx / sealed state, history entry with **slug**, copy link to /certification/{slug}.
3. Open /certification/{slug} in a **logged-out** browser (or incognito): record loads, citation/evidence sections sane, revoked vs active copy matches record state.

Stream 2 — Verify same org (Account A)

1. As **Account A** on /realitycheck, upload the **same original file** → expect **match** and verdict **Official** (certifier org matches verifier org).
2. Confirm persisted run appears in **Scribe** (/scribe) and that **Control Center** widgets still load (defense score, gates, telemetry cards) without errors.
3. If UI exposes a link/slug to a verification artifact, open /verify/{slug} logged out → public page matches result.

Stream 3 — Cross-org verify (core requirement: “authen-testnet certifies → X verifies”)

1. Keep certification from Stream 1 (Org A).
2. Sign in as **Account B** (different org), workspace valid.
3. On /realitycheck, upload the **same certified file** → expect **match** with verdict **External** (not Official).
4. Log out (or use incognito) → same upload via /realitycheck if the flow allows signed-out verify API calls; if not, use **public** verify page only where applicable. **Minimum bar:** cross-org works for **signed-in Org B**; note in doc if anonymous verify is supported on your deployment.
5. Record **slug** from verification public page /verify/{slug}; confirm **Account A** can open that URL without being in Org B (public route).

Stream 4 — Reverse direction (“vice versa”)

1. As **Account B**, certify a **different** file (not used in Stream 1).
2. As **Account A**, verify that file → expect **match** + **External**.
3. Confirm /certification/{slug} for B’s record is readable logged out.

Stream 5 — Public and embed artifacts

1. From a known /verify/{slug}, open /embed/{slug} — layout fits iframe; no auth wall.
2. Share links (/certification/..., /verify/...) across devices/browsers; confirm **no org leakage** in the sense of **wrong record** (you asked to skip deep leakage regression—still note **spot-check** that slugs 404 for bogus values).

Stream 6 — Settings and org admin (Account A admin)

1. /setup: save issuer/agency fields, gate targets; create or adjust **team/client**; assign **Account C** to a client and confirm Stream 0 no-access clears after refresh.
2. Re-run a quick certify to ensure write scope still works after assignment changes.

Stream 7 — Negative and edge cases (short list)

- **No match:** unrelated file → **No Record** (or equivalent), Scribe/logs consistent.
- **Revoked record (if applicable):** certify then revoke in-product; verify shows appropriate revoked banner ([getCertificationUi](app/certification/[slug]/page.tsx) / verification display components).
- **Invalid workspace params:** /certify?clientId=invalid → redirect behavior per lib/workspace-gate.ts (clean fallback, no crash).

Stream 8 — Sign-off

- Table: Stream | Tester | Date | Env | Pass/Fail | Notes (slug / tx / screenshot link).
- Explicit **cross-org row** marked mandatory for release.

Optional diagram for the doc (mermaid)

```
flowchart LR
  subgraph orgA [Org_A_authen_testnet]
    CertA[Certify]
    VerifyA[Verify]
  end
  subgraph orgB [Org_B_random_X]
    CertB[Certify]
    VerifyB[Verify]
  end
  Public[Public_slugs_cert_verify_embed]
  CertA --> Public
  CertB --> Public
  VerifyB -->|"match_External"| CertA
  VerifyA -->|"match_Official"| CertA
```

Implementation note (post-approval)

Single new file under docs/, ~4–6 pages in markdown: checklists with pass/fail columns, copy-paste URL templates, and pointers to the three Clerk personas. No code changes unless you later want CI to link this doc from README.

Tier 1 testing and assurance plan

Scope (what “Tier 1” means here)

Aligned with the earlier framework, this tier covers:

1. **Security** — AuthZ model and internet-facing exposure; secrets and dependency hygiene; one structured assessment (SAST/DAST plus manual review or external pentest).
2. **Privacy** — Data inventory, flows, retention; access to sensitive reads; encryption expectations; **one** backup/restore drill.
3. **Legal / product** — External claims (UI, APIs, templates) match what the system actually proves; no overclaim in crisis/PR copy.

Out of scope for Tier 1: SOC 2, formal ISO, full WCAG audit, load testing at scale (those were Tier 2+).

Architecture touchpoints in this repo

- **Route protection**: middleware.ts marks /verify, /embed, and /certification as public and protects /setup, /scribe, /control-center. *API routes under /api/ are not listed as public or protected**, so their Clerk behavior defaults need explicit verification during testing.
- **Public-style APIs**: app/api/verify/route.ts (hash lookup), app/api/verify/file/route.ts (multipart upload + verification pipeline; auth() is invoked **after** heavy work, mainly for orgId / verdict badge — intentional or not must be validated).
- **Authenticated API**: app/api/certify/dna/route.ts uses Clerk auth().
- **High-risk dev surface**: app/api/test-graph/route.ts is labeled temporary and returns reconstructed seal data — confirm it is **disabled or removed in production** as part of Tier 1.
- **CI today**: .github/workflows/ci-actions.yml runs lint, Vitest action tests, parity gates; **no npm audit, SAST, or secret scanning** in CI yet.

```
flowchart LR
  subgraph public_surfaces [Public_surfaces]
    VerifyPages["/verify /embed"]
    ApiVerify["/api/verify GET"]
    ApiVerifyFile["/api/verify/file POST"]
  end
  subgraph protected_app [Protected_app]
    Scribe["/scribe"]
```

```
  Control["/control-center"]
  ApiCertify["/api/certify/dna"]
 end
 Clerk["Clerk middleware"]
 Clerk --> protected_app
 public_surfaces --> Tier1Review["Tier1 review"]
```

Chronological phases (agent vs team)

Work is ordered so **discovery and blocking risks** come first, then **privacy and operations**, then **external assurance and sign-off**.

Phase 1 — Scope and environment (days 1–2)

Who	Actions
You / team	Name environments (e.g. staging vs prod), freeze a **release candidate** build for testing, assign an owner for security findings and for privacy (DPO or delegate). Decide whether Tier 1 pentest is **internal + tools** only or **external vendor** (PH firms often expect a written report for procurement later).
Agent	Produce a **single inventory doc** (can live in .cursor/ or docs/ per your preference): all app/api/**/route.ts handlers, HTTP methods, and intended auth (public / session / org). Flag mismatches with middleware.ts. Explicitly call out app/api/test-graph/route.ts and app/api/geocode/route.ts (third-party call, logging).

Phase 2 — Security: application and API review (days 2–5)

Who	Actions
Agent	Deep read of org scoping in app/actions/client-scope.ts and Clerk usage across app/actions/*.ts; draft **abuse cases** (cross-org access, slug guessing on embed, unauthenticated flood of POST /api/verify/file, metadata leakage in JSON responses). Add or extend **automated tests** where cheap and high-value (e.g. Vitest for API route handlers that already have patterns in app/api/certify/dna/route.test.ts).

You / team	**Manual threat modeling** session (1–2 hours): assets (client lists, drafts, verification records), actors (PR firm staff, journalists, hostile third parties), trust boundaries (browser → Next → Neon → Clerk). **Decide** intended policy: should POST /api/verify/file be anonymous, rate-limited, or authenticated? Implement policy is **team + agent in Agent mode**; Tier 1 only requires the **decision and verification**.

Phase 3 — Security: dependencies, secrets, and scanning (days 3–7, parallel with Phase 2)

Who	Actions
Agent	Run and summarize **npm audit** (or npm audit --production) on a clean install; document top issues and upgrade paths. Optionally propose **CI steps** (e.g. npm audit --audit-level=high with documented exceptions, or Dependabot config) — implementation in Agent mode after you approve policy. Grep-style **secret hygiene** checklist: no keys in repo, .env.local patterns, Clerk publishable vs secret keys documented.
You / team	Enable **GitHub Dependabot** (or equivalent) at org/repo level if not already on. **Rotate** any credential that was ever exposed. **Run** one SAST tool you are comfortable operating (e.g. GitHub CodeQL on default JS, or Semgrep rules for OWASP) — the agent can interpret SARIF/output and map findings to files; **you** run the tool in your org's approved toolchain.

Phase 4 — Security: DAST / pentest on internet-facing surfaces (days 5–14)

Who	Actions
You / team	Point **DAST** or **pentest** at **staging** (preferred) or production: public pages (/verify, /embed, /certification), public APIs (/api/verify, /api/verify/file), login flows, and Clerk-hosted flows. Collect a **written report** (even if internal).
Agent	Turn pentest findings into **tracked remediation items** (file + suggested fix), and verify fixes with tests where applicable. **Not** a substitute for the pentester.

Phase 5 — Privacy: data map and technical controls (days 5–12)

Who	Actions

You / team	Lead **data map** workshops: categories of personal data (PR staff, end subjects in media assets, influencers, etc.), purposes, lawful basis under **Philippine DPA** (with **local counsel** — the agent does not provide legal advice). Approve **retention** and deletion expectations per data category.
Agent	Draft a **technical appendix** to the data map from code and schema: tables touched by verification/logging (e.g. via scripts/sync-database.ts and actions), fields that may hold PII, subprocessors (Clerk, Neon, Vercel/Netlify, any AI or blockchain providers you use). List **where logs are written** (console.error, external APM if any).

Phase 6 — Privacy: encryption, access logging, backup drill (days 8–14)

Who	Actions
You / team	Confirm **TLS** everywhere for user-facing hosts; confirm **Neon** / dashboard **encryption at rest** per vendor docs; define **who** may access production data (founders, support) and document it. Execute **one backup restore drill** (Neon PITR or snapshot — follow Neon console docs); record **RTO/RPO** actually achieved.
Agent	If you want **access logs** for sensitive server actions, the agent can propose minimal structured logging patterns in code **after** you define policy (what to log, what not to log — avoid logging file contents or unnecessary PII).

Phase 7 — Legal / product alignment (days 10–18, parallel)

Who	Actions
You / team	**Philippine counsel**: privacy notice, DPA with PR firms (controller/processor roles), breach notification playbook aligned with NPC expectations. **PR/comms**: review crisis templates (e.g. docs/crisis_brief_and_witness_statement_templates.md) so language does not overclaim provenance or legal admissibility.
Agent	**Consistency pass**: compare user-visible strings and API payloads (e.g. proof.authority in app/api/verify/route.ts) against internal honesty docs such as .cursor/plans/technical_assumptions_limitations_and_claims.md and docs/INVESTOR_BRIEF.md where relevant; output a **gap list** (claim → evidence in code or missing).

Phase 8 — Exit criteria and sign-off (days 14–21)

Who	Actions
You / team	Sign-off checklist: (a) no critical/high open issues from pentest/DAST without accepted risk, (b) data map + retention v1 approved, (c) backup drill completed, (d) legal/privacy docs v1 published or ready for pilot clients, (e) test-graph (or equivalent) not exposed in prod.
Agent	Maintain a **single remediation backlog** linked to phases above; re-run automated tests and audit summary after fixes.

Deliverables checklist (Tier 1 complete when…)

- API and middleware **auth model** documented and tested against intent (including /api/verify/file).
- **Temporary or debug routes** verified absent or gated in production.
- **npm audit** (and optionally CI gate) addressed or risk-accepted in writing.
- At least one **structured security assessment** (SAST + DAST and/or pentest report) with **closed or accepted** critical/high issues.
- **Data map v1** + subprocessors list + retention outline (legal owner sign-off).
- **Backup restore drill** completed once with notes.
- **Claims review**: UI/API/templates aligned with limitations doc; counsel reviewed external-facing legal text.

Notes

- **Agent mode** is required for code changes, CI edits, and new tests; this plan is structured so you can execute Phases 1–2–3 with an agent in a follow-up session.
- Philippines **NPC** registration and detailed **lawful basis** are **team + counsel**; the agent supports only technical documentation and checklists.

Tier 2 testing and assurance plan

Relationship to Tier 1

Tier 1 establishes baseline security, privacy v1, legal/claims alignment, and one backup drill. **Tier 2** assumes those are in place (or in flight with accepted gaps) and focuses on **scale under stress**, **operational readiness**, **procurement-grade hygiene**, and **inclusive public surfaces**.

If Tier 1 exit criteria are not met, run Tier 2 only on **non-production** or accept that performance and IR tests may surface issues that overlap Tier 1 (e.g. unauthenticated abuse of public APIs).

Scope (what Tier 2 means here)

1. **Performance and resilience** — Spike and soak patterns for “incident day” traffic; latency SLOs for critical paths; validation of **rate limiting**, **queues**, or **caching** on expensive work (multipart verification, DNA extraction, external RPC/blockchain reads).
2. **Operational maturity** — Incident response runbooks, on-call expectations, **vendor DPAs** / subprocessor governance, breach comms templates (NPC-aligned **as counsel defines**), disaster recovery beyond a single backup drill (e.g. regional failure scenarios at a tabletop level).
3. **Accessibility** — WCAG-oriented testing on **public** and high-traffic **client** flows (/verify, /embed, key marketing or certification pages as applicable), with remediation backlog prioritized by severity and procurement needs.

Explicitly out of scope for Tier 2 (Tier 3 / enterprise): SOC 2 Type II audit, ISO 27001 certification, full 24/7 NOC, multi-region active-active.

Architecture touchpoints in this repo

- **Large payloads**: next.config.ts sets **50MB** server action body and middleware client body limits—load tests must include **realistic file sizes** and abuse cases (oversized uploads, concurrent uploads) to observe CPU/memory and Neon write pressure.
- **Hot paths**: app/api/verify/file/route.ts runs verification + logging; public app/api/verify/route.ts is hash lookup—different load profiles. [app/embed/[slug]/page.tsx](app/embed/[slug]/page.tsx) may drive **bursty read traffic** from third-party sites; internal planning already calls out **RPC rate limits** when many embeds resolve on-chain data (see .cursor/plans/crisis_brief_witness_chain_decode_implementation.md for the caching/risk framing).
- **CI**: .github/workflows/ci-actions.yml does not run performance or a11y gates yet—Tier 2 can add **optional** scheduled jobs (k6, Lighthouse CI, axe) once baselines exist.

```
flowchart TB
  subgraph load [Load_drivers]
    EmbedBurst["Many embed iframes"]
    VerifyFile["POST verify/file"]
    VerifyGet["GET api/verify"]
  end
  subgraph deps [Dependencies]
    App["Next.js app"]
    Neon["Neon DB"]
    Rpc["Blockchain RPC"]
  end
  load --> App
  App --> Neon
  App --> Rpc
```

Chronological phases (agent vs team)

Phase 1 — Baselines and SLOs (week 1)

Who	Actions
You / team	Define **SLO targets** for pilot PR firms (e.g. p95 latency for embed page, verify API, Scribe submit if applicable). Pick **staging** URL and **test windows** so load tests do not surprise production neighbors. Assign an owner for **observability** (where logs/metrics live: Vercel/Netlify, Neon, Clerk dashboards).
Agent	Document **current critical paths** and dependencies in a short runbook appendix: which routes call DB, external geocoding (app/api/geocode/route.ts), graph/RPC, PDF/image pipelines. Propose **minimal metrics** to watch during tests (error rate, p95, DB connection count, function duration).

Phase 2 — Performance test design (week 1–2)

Who	Actions
You / team	Approve **scenarios**: (a) embed burst (many parallel GETs for a few slugs), (b) steady verify traffic, (c) spike of POST /api/verify/file with representative files, (d) optional soak (30–120 min). Decide **acceptable cost** (Neon compute, serverless invocations, RPC provider quotas).

Agent	Draft **k6 or Artillery** scripts (or equivalent) checked into scripts/ with README: environment variables, staging base URL, **no secrets in repo**. Add **guardrails** (max VUs, duration caps). If you use a paid load tool, agent outputs **importable configs**; **you** run them in your approved account.

Phase 3 — Execute load and resilience tests (week 2–3)

Who	Actions
You / team	Run load tests against **staging** (preferred). Capture screenshots or exports from dashboards. If production testing is required, do it in a **maintenance window** with rate caps and legal/comms awareness.
Agent	After each run, **summarize results** vs SLOs, file GitHub issues for regressions. If bottlenecks are clear in code (missing cache headers, N+1 queries, unbounded RPC), propose **targeted fixes** (implementation in Agent mode).

Phase 4 — Rate limiting, cost controls, and caching (week 2–4, parallel)

Who	Actions
You / team	Decide **policy**: per-IP vs per-org limits, free vs paid tiers, WAF (e.g. Cloudflare) vs app-level limits. Approve **budget alerts** on Neon, RPC, and hosting.
Agent	Implement or sketch **middleware / edge rate limits** and **caching** for embed and read-heavy paths per approved design; add **tests** where behavior is deterministic. Align with existing internal notes on **embed + RPC** behavior.

Phase 5 — Incident response and vendor governance (week 3–5)

Who	Actions
You / team	Write **IR runbook** v1: severity levels, who calls whom, customer comms template, NPC breach steps **per Philippine counsel**. Maintain **subprocessor register** and execute **DPAs** with Clerk, Neon, host, RPC, any AI vendors. Run a **tabletop** (simulated DB outage or Clerk outage).
Agent	Produce **technical IR appendix**: how to rotate keys, disable routes, read Neon PITR, rollback deploy, feature flags if any. Keep it as a single doc linked from README or docs/ per your conventions.

Phase 6 — Accessibility (WCAG-oriented) (week 4–6)

Who	Actions
You / team	Choose **scope** (e.g. /embed/:slug, /verify/:slug, landing pages). Decide target level (**WCAG 2.1 AA** is common for procurement). Optionally engage a **short a11y audit** from a specialist firm.
Agent	Run **automated scans** (axe, Lighthouse) in CI or locally and fix **clear** issues: focus order, form labels, contrast, heading hierarchy, embed iframe title if applicable. Manual testing (keyboard, screen reader) remains **team + specialists**.

Phase 7 — Regression gates (week 5–6)

Who	Actions
You / team	Decide which Tier 2 checks are **merge-blocking** vs **weekly scheduled** (performance budgets drift; full k6 may be too slow for every PR).
Agent	Wire **optional CI jobs**: Lighthouse CI with thresholds, or k6 smoke with low VUs; document how to interpret failures and when to waive.

Phase 8 — Tier 2 sign-off (week 6+)

Who	Actions
You / team	Sign-off: SLOs met or risks accepted in writing; IR runbook + tabletop done; DPAs current; a11y scope addressed or gap list with timeline for PR/government pilots.
Agent	Consolidate **evidence pack** (test outputs, before/after metrics, closed tickets) for investor or enterprise conversations—technical annex only.

Deliverables checklist (Tier 2 complete when…)

- Documented **SLOs** and **load-test results** for agreed scenarios (staging or controlled prod).
- **Rate limiting / cost / caching** decisions implemented or explicitly deferred with rationale.
- **IR runbook v1** + **tabletop** completed; **subprocessor list** and **DPAs** in good standing for Tier 2 vendors.
- **WCAG-oriented** pass on agreed public surfaces, with known issues tracked.
- Optional: **CI regression hooks** for perf/a11y smoke tests.

Notes

- Tier 2 is **not legal advice**; NPC breach wording and DPAs are **team + counsel**.
- The **agent** implements code and automation; **you** run production-adjacent load tests, approve policies, and sign contracts.

Tier 3 testing and assurance plan

Relationship to Tiers 1 and 2

- **Tier 1** — Security/privacy/legal baseline, pentest, data map, backup drill, claims alignment.
- **Tier 2** — Performance/SLOs, IR runbooks, DPAs, WCAG-oriented passes, optional CI perf/a11y smoke.
- **Tier 3** — **Third-party attestation** (or equivalent), **ongoing evidence** for controls, and **repeatable** vendor diligence (questionnaires, annual pen tests, access reviews) so **PR firms, government, and large NGOs** can rely on a **documented, audited** program—not only point-in-time tests.

Tier 3 is usually driven by **a customer contract**, **RFP**, or **procurement gate**, not by curiosity. Start Tier 3 when you have a **named** requirement (e.g. “SOC 2 Type II within 12 months”) or a **lost deal** citing lack of report.

Scope (what Tier 3 means here)

1. **Formal assurance** — Most often **SOC 2 Type II** (Trust Services Criteria: Security; optionally Availability, Confidentiality, Privacy). Alternative: **ISO 27001** certification if buyers in your pipeline ask for it. **SOC 2 Type I** is a shorter “point-in-time” option; many enterprises still want **Type II** for a period of operating effectiveness.
2. **Continuous control operations** — Evidence that controls run **all year**: change management, access reviews, vulnerability management, incident tracking, vendor reassessment, backup testing on a **cadence** (not one-off).
3. **Enterprise testing rhythm** — **Annual** (or major-release) penetration test, **quarterly** internal security reviews, **ongoing** dependency/supply-chain process tied to what auditors sample.
4. **Business continuity at commitment level** — Documented **RTO/RPO** you are willing to **stand behind** in contracts; multi-step failover or warm standby if you promise high availability (optional and costly—scope explicitly).

Out of scope unless a customer pays for it — FedRAMP, PCI Level 1, dedicated single-tenant infra, 24/7 SOC with contractual response times.

Architecture and evidence touchpoints (this repo)

Auditors and customers care that **technical controls are real**, not only policies. Useful anchors:

- **Change control**: Git history, PRs, protected branches, .github/workflows/ci-actions.yml (lint + tests). Tier 3 typically requires **branch protection** and **required checks** on main.

- **Build and deploy**: Document who can deploy production (hosting provider roles: Vercel/Netlify/GitHub).
- **Secrets**: No secrets in git; rotation evidence; Clerk/Neon/hosting API keys scoped minimally.
- **Monitoring and IR**: Links to dashboards, ticketing for incidents, postmortems (even lightweight).
- **Subprocessors**: Clerk, Neon, RPC, host—aligned with Tier 2 DPA list but **reviewed on a schedule** with evidence.

The **agent** cannot sign attestations or talk to auditors; it can **map controls to repos and automation** and **implement** guardrails (CI, docs-as-code for runbooks).

```
flowchart LR
  subgraph people [Organization]
    Mgmt["Management"]
    Eng["Engineering"]
  end
  subgraph evidence [Evidence_systems]
    Git["Git plus CI"]
    Tickets["Issue_tracker"]
    Comply["GRC_or_spreadsheet"]
  end
  subgraph external [External]
    Auditor["Auditor"]
    Customer["Enterprise_buyer"]
  end
  Eng --> Git
  Mgmt --> Comply
  Auditor --> Comply
  Customer --> Auditor
```

Chronological phases (agent vs team)

Phase 1 — Trigger and framework choice (week 0–2)

Who	Actions
You / team	Capture **exact buyer language** (SOC 2 vs ISO, which TSC criteria, need for Privacy criterion). Set **budget** (auditor + optional GRC tool: Vanta, Drata, Secureframe, or manual). Name an **internal owner** (often CFO/COO + eng lead).
Agent	None critical; optionally produce a **one-pager** comparing SOC 2 Type I vs II vs ISO 27001 **from a startup ops perspective** (timelines, cost order-of-magnitude research is team-led).

Phase 2 — Readiness / gap assessment (weeks 2–6)

Who	Actions
You / team	Complete vendor questionnaire or GRC **gap assessment** with honest answers on HR (background checks), device policy, MFA everywhere, production access.
Agent	**Technical gap list**: branch protection status, CI coverage, secret scanning in CI, MFA on GitHub, list of production env vars and where they live, inventory of **who has admin** on Clerk/Neon/host. Map each to likely SOC 2 control IDs (CC6, CC7, etc.) as a **draft** for your compliance lead—not legal advice.

Phase 3 — Policy and control design (weeks 4–10)

Who	Actions
You / team	Approve **policy pack**: acceptable use, access control, change management, vendor management, incident response, BCP/DR, risk assessment cadence. HR pieces: onboarding/offboarding checklist.
Agent	Maintain **technical control narratives** that reference this repo: how deploys work, how tests block merge, how backups work (Neon), encryption in transit (TLS) references. Keep **diagrams and links** current so GRC uploads are copy-pasteable.

Phase 4 — Implement and instrument controls (weeks 6–16, overlaps)

Who	Actions
You / team	Enforce **MFA** and **SSO** (if required) on GitHub, Google Workspace, Clerk dashboard, cloud consoles. Restrict production access to named accounts. Start **ticketed** change records for prod changes if not already.
Agent	Implement **automation evidence**: required status checks, optional **CodeQL** or Semgrep in CI, Dependabot alerts export, **documented** release process in README. Add **scheduled** jobs that prove recurring tasks (e.g. quarterly npm audit report artifact in CI artifacts—policy-dependent).

Phase 5 — Observation period (SOC 2 Type II: ~3–12 months)

Who	Actions

You / team	Operate **normally** while collecting evidence: access reviews (quarterly), vendor reviews, incident log, backup tests **on schedule**, pen test remediation tracked in tickets. **No "audit theater"**—auditors sample reality.
Agent	Each quarter, help **export** technical evidence: CI run history, merged PR list with reviewers, dependency scan summaries, infra change log from git. Format as **auditor-friendly** appendices (dates, scope, owners).

Phase 6 — Pre-audit and testing window (final 4–8 weeks before fieldwork)

Who	Actions
You / team	Hire **auditor**; complete readiness review. Optionally run **internal mock audit**. Commission **annual penetration test** if not already in the observation window; remediate **critical/high** before fieldwork when possible.
Agent	Fix findings that are **code/config** (with your prioritization). Add **tests** proving fixes. Prepare **demo environment** walkthrough script for auditor technical sessions.

Phase 7 — Audit fieldwork and report (4–6 weeks typical)

Who	Actions
You / team	Respond to requests, pull HR and vendor evidence, attend interviews. Review draft report and **management letter** exceptions.
Agent	Support **evidence pulls** (screenshots of settings, redacted configs, CI links). Does **not** replace compliance owner.

Phase 8 — Continuous compliance and renewal (ongoing)

Who	Actions
You / team	**Annual** SOC 2 renewal or ISO surveillance audits; **annual** pen test; **quarterly** access reviews; update risk register. Renew **customer-facing trust page** (report under NDA if needed).
Agent	When controls or stack change, **update technical narratives and diagrams**; adjust CI checks to match new policies; help answer **customer security questionnaires** by mapping answers to implemented controls (draft responses for your review).

Deliverables checklist (Tier 3 complete when…)

- **Signed** SOC 2 Type II report (or ISO 27001 certificate) in hand, with **scope** (systems and boundaries) matching what you sell to PR firms.
- **Observation-period evidence** archived (access reviews, incidents, changes, backups, vendor reviews).
- **Annual penetration test** program established with remediation tracking.
- **Customer-facing package**: trust page or one-pager, subprocessors, DPA, and process for **NDA-gated** report sharing.

Notes

- **Philippines NPC** registration and DPA compliance remain **parallel legal tracks**; SOC 2 does not replace local privacy law—it **supports** enterprise trust.
- Tier 3 is **expensive in time**; defer until a **concrete** enterprise or government pipeline requires it.
- The **agent** accelerates **technical mapping and automation**; **you** own budget, HR policies, auditor relationship, and sign-offs.